BeachWell: Predicting Student Workload from Course Deadlines and Recommending Recovery

Course project, CECS 458/558 (Dr. Moon), California State University, Long Beach, Fall 2026. In progress; this summary reflects the state on October 29, 2026.

Vo Hong Vy Le, with Deborah Shaw

My role: backend, AI components and evaluation (workload model, personal pace model, planner, event recommender, safety screen, evaluation harness). **Deborah Shaw:** web app, campus event research, labeling and reports.

Students see due dates in their learning management system, but not how much work a week actually holds, so crunch weeks are noticed too late. BeachWell reads a student's Canvas calendar feed, estimates the hours behind each deadline, and gives every week a transparent workload score from 0 to 100 with reasons. It then builds a constrained study plan that moves work into lighter weeks, recommends campus de-stress events that fit the student's free time, and gives counseling and student services an anonymous view of when whole groups are heading into overload. Each component is evaluated against a simple non-AI baseline.

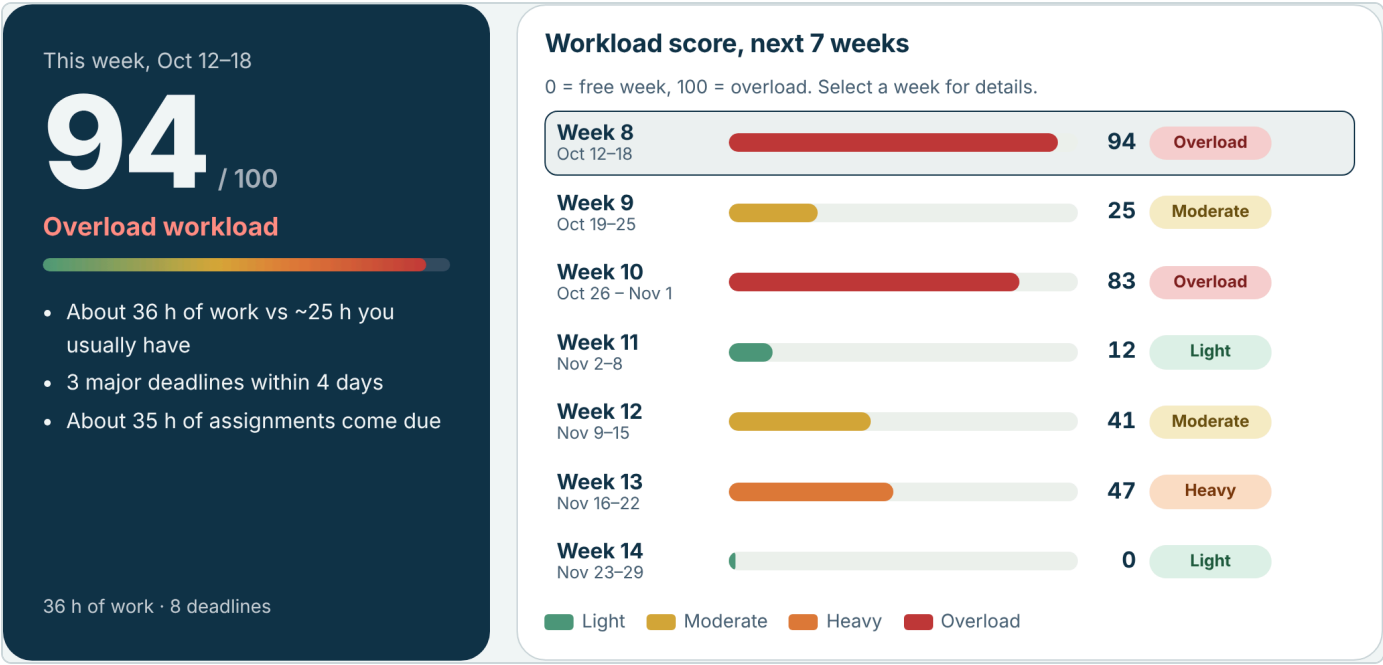


Figure 1. Workload screen for the demo student in week 8: this week's score with the reasons behind it, and the scores of the next seven weeks.

Problem

Workload is a leading source of stress for college students, and deadlines from different courses tend to collide in the same few weeks. Calendars list deadlines but say nothing about effort, and campus support (counseling workshops, recreation, peer programs) is easy to miss when a student needs it most. The project asks whether a student's own course calendar is enough to forecast overload weeks early, and to turn that forecast into actions: a plan, a break, or a campus program.

System and methods

1. **Effort estimation.** A large language model (Gemini, few-shot prompt with JSON output and validation) classifies each assignment and estimates its hours, with a rule-based fallback using title cues, page counts and point values. A per-type calibration learned from team-labeled examples corrects the hours, shrunk toward the overall ratio when a type has few labels.

2. **Workload score.** A transparent index combining hours relative to the student's usual study capacity, the density of major deadlines within any four days, deadline pressure, and recent sleep and stress check-ins, mapped to four levels with generated reasons. "What-if" counterfactuals re-run the score when a task starts earlier or the plan is followed.
3. **Personal pace.** An empirical-Bayes model of each student's log(actual / estimated) hours per assignment type, shrunk toward the student's overall ratio, updates estimates as the student logs time.
4. **Planner.** Earliest-deadline-first scheduling into blocks of at most 90 minutes with breaks, avoiding classes, meals and due times, within daily study limits; a rule checker verifies every plan.

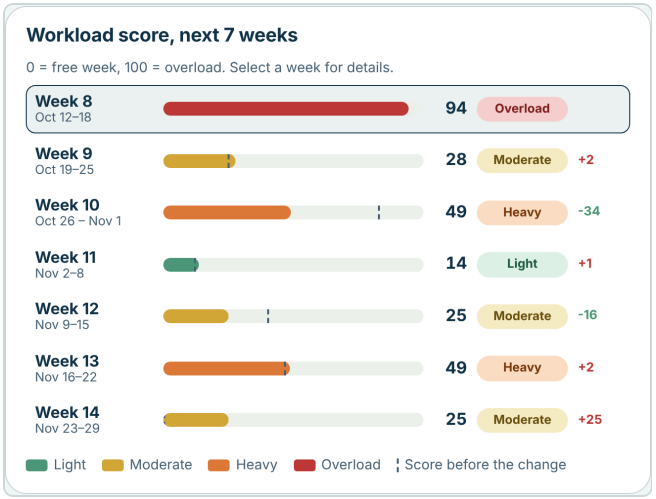


Figure 2. What-if: following the study plan, week 10 drops from Overload to Heavy (−34) and week 12 from 41 to 25; dashed marks show each week's score before the change.

5. **Event recommender.** Hybrid retrieval combining embedding similarity, literal word overlap, and event profiles learned from labeled sample students (relevance feedback); a free-time filter, a crowd-aversion rule and cross-week diversity, over a catalog of 50 real campus programs (22 with times confirmed on official pages); online, the LLM re-ranks the shortlist and may only return listed event IDs.
6. **Safety and privacy.** Every free-text note passes a recall-first screen (rules or LLM); a fixed, human-written message routes the student to the 988 Lifeline and campus counseling. The campus view aggregates opted-in students only and suppresses any group smaller than ten.

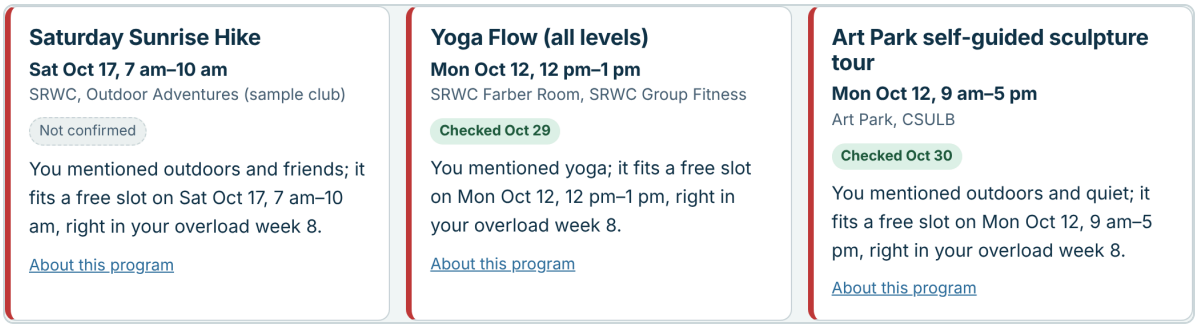


Figure 3. Recommendations for an overload week, placed in the student's free time, with reasons that quote what the student said helps them recharge.

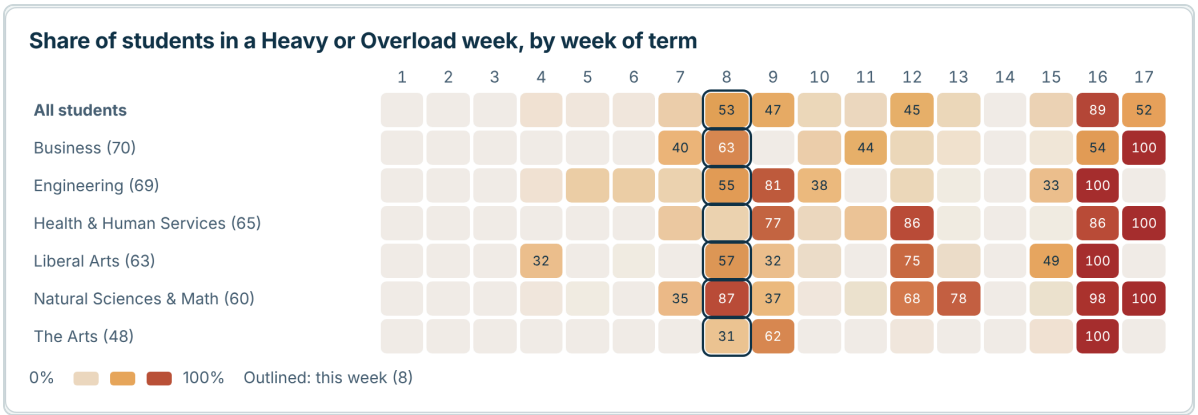


Figure 4. Campus view for counseling and student services (375 opted-in students in a synthetic cohort): share of students in a Heavy or Overload week, by college and week of term. Groups under 10 are hidden.

Evaluation

All results below are from offline mode (rules and hashed embeddings, no API key), so they are fully reproducible with `python eval/evaluate.py`. Anything learned from labels is tested leave-one-out.

Component and data	Baseline	BeachWell
Hour estimates, mean absolute error (60 assignments labeled by a teammate)	2.99 h (type keyword); 2.63 h (rules)	**1.92 h** (rules + learned calibration)
Finding the planted crunch week, F1 (40 synthetic semesters with decoy weeks)	0.67 (deadline count)	**1.00**
Estimate error after learning pace, weeks 9–17 (40 simulated students)	1.28 h (no pace model)	**0.68 h**
Planner: busiest week and overload weeks per student (20 synthetic semesters)	36.0 h; 2.65 weeks	**27.4 h; 1.05 weeks**, 0 rule violations
Event picks for 24 sample students, 20 programs: Precision@3, Hit@1, MRR	0.38, 0.63, 0.74 (keyword) 0.44, 0.71, 0.82 (first version)	**0.54, 0.88, 0.93**
Safety screen, crisis recall on a teammate's unseen set of 15 help-needed and 15 everyday-stress messages	–	first version caught **6/15**; see below

Labels currently come from one annotator; a second annotator's labels will add an agreement measure (Cohen's kappa). Three of the six experiments use synthetic data. Gemini runs, a second safety test set and a usability study with 8–10 classmates (System Usability Scale) are scheduled before the final report.

What I learned

- **Held-out evaluation matters more than the model.** The safety rules scored 100% recall on the set they were written against, and 40% on a teammate's fresh set. I turned the misses (hopelessness, "ending everything", threats to others) into general patterns, checked them against everyday exaggerations, and reclassified that set as development data. A third set, written without seeing the rules, is now the honest test.
- **Error analysis beats tuning.** In the recommender, a generic "relax, stress relief" phrase added to every query made one stress workshop appear in almost every top 3 (a hub effect). Removing it and adding light stemming raised Hit@1 from 0.71 to 0.88 before any learned component; learned event profiles then improved Precision@3 further.
- **Small data needs restraint.** With 24 labeled students and 60 labeled assignments, I used shrinkage and leave-one-out evaluation rather than fitted weights, and kept the scoring formula interpretable so every number shown to a student can be explained.

Next steps

Measure the Gemini components against the offline ones; collect a second set of labels and report agreement; run the classmate usability study; and, beyond the course, study whether early workload forecasts change when students start work and how stressed they report feeling, using consented, de-identified data.

Built with Python (Starlette, SQLite, NumPy), plain JavaScript, and the Gemini API. Code, tests and the evaluation harness are available on request.